

Identifying Public Data in Constant-Time Programs with Symbolic Execution

Yavor Litchev (ylitchev@stanford.edu) & Jane Nguyen (janen9@stanford.edu)

Motivation

- Timing and microarchitectural side channels can reveal secret material
 - Secret-dependent control flow and secret-dependent data flow can result in different observable behaviors [1]
- Cryptographic and security-critical code require appropriate protections
- Constant-time programming avoids side channels by only passing secrets to safe instructions, creating *operand-independent* hardware resource usage
 - Assumes sequential execution of programs
- Speculative execution undermines CT programming
 - Microarchitectural state modified in transient execution with misprediction, even if instruction is not architecturally retired
- Unsafe instructions can leak secret information through transmitters
- Apply mitigations, but only where secrecy is *required*
 - Security guarantees often come with high performance cost from conservative protections
 - Public vs. Potentially Secret data

```
int foo(int *p, int secret, int x) {  
    int y = x + 1;  
    int z = *p;           // load address transmitter  
    if (secret & 1) {      // branch condition transmitter  
        y = y + z;  
    }  
    *p = y;               // store address transmitter  
    return y;  
}
```

Control-flow and Memory Transmitters

```
if (false but mispredicts as true)  
    multiply R1, R2  
multiply R3, R4
```

Variable-Time Transmitters [3]

Related Work

Defenses against Speculative Execution Attacks

- Naively, protect *all* registers and memory accesses
- Speculative Load Hardening (SLH)
 - Mask values misspeculated, use OR to harden/poison access
 - High performance overhead from extra bitwise operations

JC taken	JC taken
.	CMOVC -1, %rcx
.	.
.	.
JMP out	JMP out
taken:	taken:
.	CMOVNC -1 %rcx
.	.
out:	out:

Bitmask Creation with SLH [2]

Recent work attempts to decrease performance overheads

- Speculative Privacy Tracking (SPT)
 - Hardware dynamically tracks execution, marking data as “public” if leaked at instruction retirement
- Declassiflow
 - Statically remove protection from program points guaranteed to leak via Non-Speculative Knowledge (implicitly declassified)
- Protean
 - Utilize ISA extension for SW-to-HW communication of which registers/memory to protect
 - Compiler infers protections, HW enforces SW-defined protections

Problem

- Constant-time paradigms are **insufficient** for guaranteeing security, preventing observable architectural leakage but leaking through microarchitectural state with speculation
- Existing defenses incur performance overheads and define “publicness” via non-speculative knowledge
 - Inserting more instructions, use additional HW resources, extend ISAs
 - Overprotection is costly, underprotection leaks secrets
- Complementary to existing literature, we:
 - Analyze programs at intermediate value granularity
 - Statically define where transmitters are
 - Determine public vs. potentially secret data based on publicness defined by dual symbolic execution
 - Leverage SMT solving for publicness determination
- Enable selective mitigation: If a value is provably public, it can be left unprotected without weakening the intended security policy

Given an LLVM Intermediate Representation (IR) and a transmitter-based leakage model, determine which registers are public at each program point in a given function.

Technical Approach

Path Enumeration

- Compose each function as a CFG with basic blocks (BBs) and edges
- Use the CFG to enumerate all possible paths through the function

Publicness Definition

- Symbolically execute each control-flow path P twice, asserting each transmitter operand is equal between each instance
- A value is public along the path if it can be assigned different values between instances
 - Leverage SMT solver to determine satisfiability of public-along-path condition
- Aggregating all paths through a program point, a value is public-at-program-point iff it is public along all paths through that point

$$\begin{aligned} \text{public along path}(P, v) &:= \text{SAT}(\text{path constraints}_A(P) \wedge \text{path constraints}_B(P) \wedge \text{transmitter equality}(P, v_A, v_B) \wedge (v_A \neq v_B)) \\ \text{public at point}(pp, v) &:= \forall \text{ paths } P \text{ through a pp: public along path } (P, v) \end{aligned}$$

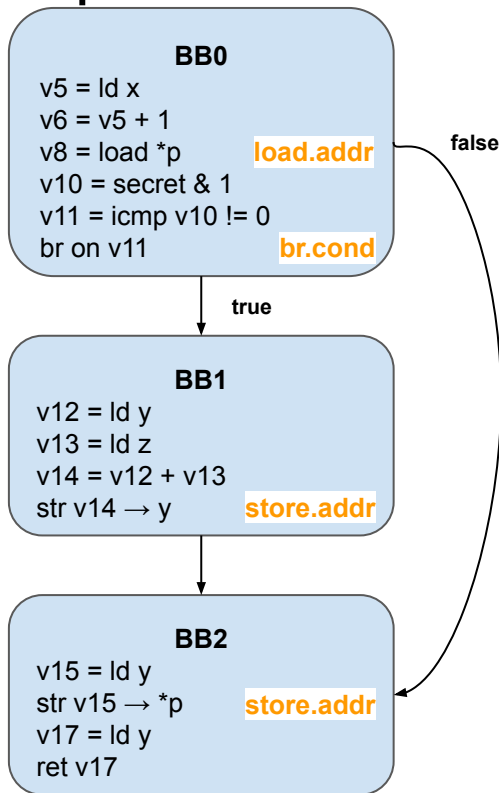
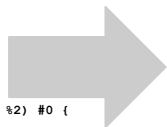
Technical Approach: Example

```
int foo(int *p, int secret, int x) {  
    int y = x + 1;  
    int z = *p;      // load address transmitter  
    if (secret & 1) { // branch condition transmitter  
        y = y + z;  
    }  
    *p = y;          // store address transmitter  
    return y;  
}
```

Function

```
define dso_local i32 @foo(ptr noundef %0, i32 noundef %1, i32 noundef %2) #0 {  
    %4 = alloca ptr, align 8  
    %5 = alloca i32, align 4  
    %6 = alloca i32, align 4  
    %7 = alloca i32, align 4  
    %8 = alloca i32, align 4  
    store ptr %0, ptr %4, align 8  
    store i32 %1, ptr %5, align 4  
    store i32 %2, ptr %6, align 4  
    %9 = load i32, ptr %6, align 4  
    %10 = add nsw i32 %9, 1  
    .  
    store i32 %21, ptr %22, align 4  
    %23 = load i32, ptr %7, align 4  
    ret i32 %23  
}
```

LLVM IR



CFG

Path Enumeration

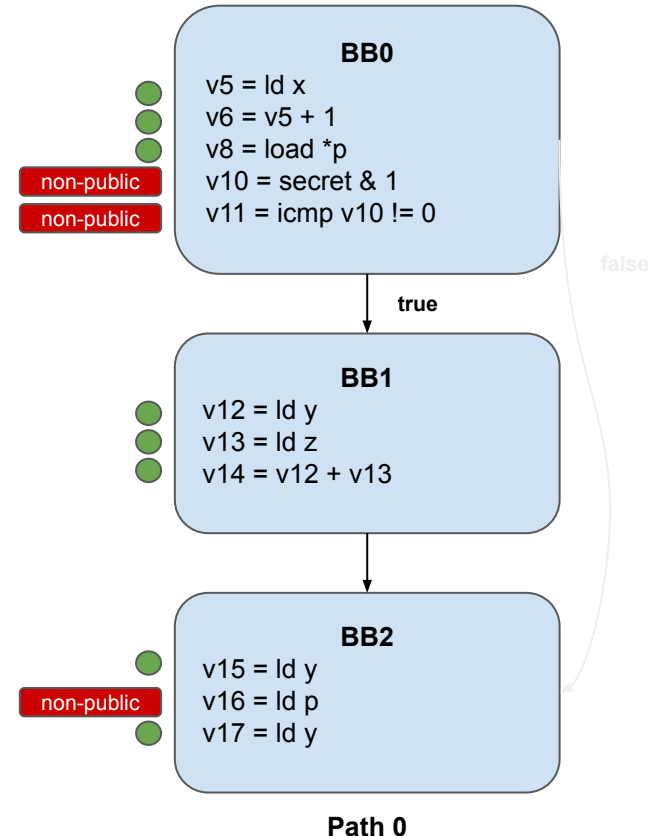
- Path 0:
BB0 → BB1 → BB2
- Path 1:
BB0 → BB2
- Transmitters: load.addr, store.addr, br.cond

Technical Approach: Example

Publicness Determination

Which registers are public vs. non-public?

- Coloring scheme represents per-path public-at-point determination
- Green = public: value can vary with transmitter equality
- Red = non-public: transmitter equality or path condition unsatisfiable
- Publicness determination enables selective declassification



Implementation: LLVM Pass

```
foo
bb_count = 3
inst_count = 28
tx_count = 17
transmitter kinds = load.addr, store.addr, br.cond
stable program points = foo:bbX:iY
```

- Identify transmitter operands
 - Load.addr, store.addr, br.cond, switch.cond, indirectbr.target, call.target, atomicrmw.addr, cmpxchg.addr, div.operand, rem.operand
 - Operand index
- Instruction trace records
 - one NDJSON object per instruction (fn, bb, pp, op, def, uses, tx) with stable IDs
- CFG path artifacts
 - Path records with path conditions and path-to-program-point mapping
 - DFS for path enumeration (path_id, bbs, decisions, path_cond, pp_seq)
 - pp_coverage enumerates paths associated with each pp for public-at-point aggregation downstream

```
int foo(int *p, int secret, int x) {
    int y = x + 1;
    int z = *p;      // load address transmitter
    if (secret & 1) { // branch condition transmitter
        y = y + z;
    }
    *p = y;          // store address transmitter
    return y;
}
```

Program

↓ clang

```
define dso_local i32 @foo(ptr noundef %0, i32 noundef %1, i32 noundef %2) #0 {
    %4 = alloca ptr, align 8
    %5 = alloca i32, align 4
    %6 = alloca i32, align 4
    %7 = alloca i32, align 4
    %8 = alloca i32, align 4
    store ptr %0, ptr %4, align 8
    store i32 %1, ptr %5, align 4
    store i32 %2, ptr %6, align 4
    %9 = load i32, ptr %6, align 4
    %10 = add nsw i32 %9, 1
```

LLVM IR

LLVM Pass →

```
{...
"pp":"foo:bb0:i12",
"op":"load",
"def":"v8", ...
"txs":[{"kind":"load.addr",
"which":0}],
...
{...
"pp":"foo:bb0:i17",
"op":"br",...
"Uses":["v11"],...
"txs":[{"kind":"br.cond", "which
":0}]
...

```

Trace NDJSON

```
{...
"path_id":0,
"bbs":["bb0", "bb1", "bb2"],
...
"path_cond": [
...
{...
"pp":"foo:bb1:i2",
"path_count":1, "path_ids":[0]}
...

```

CFG NDJSON

Implementation: Publicness Inference Pipeline

$$\text{public along path}(P, v) := \text{SAT}(\text{path constraints}_A(P) \wedge \text{path constraints}_B(P) \wedge \text{transmitter equality}(P, v_A, v_B) \wedge (v_A \neq v_B))$$

$$\text{public at point}(pp, v) := \forall \text{ paths } P \text{ through a } pp: \text{public along path}(P, v)$$

- Parsing and path construction
 - Join trace and CFG NDJSON artifacts into PathBundle(path, insts) for each path
- Dual symbolic execution
 - Symbolically execute same path twice (execution A and B)
 - Add transmitter equality constraints for each transmitter operand and path constraints for both executions
 - For each SSA on the path, use Z3 solver to check SAT ($v_a \neq v_b$), indicating *public*
- Aggregation to public at program point
 - For each (fn, pp, value), collect per-path booleans $\rightarrow$ public iff public on all paths through point (True)

Example Path chosen for SMT Query

Path 0: BB0 $\rightarrow$ BB1 $\rightarrow$ BB2

Path Constraints:

PC_A: $v_{11} == 1$

PC_B: $v_{11} == 1$

Constraints

Path Constraints:

PC_A: $v_{11} == 1$

PC_B: $v_{11} == 1$

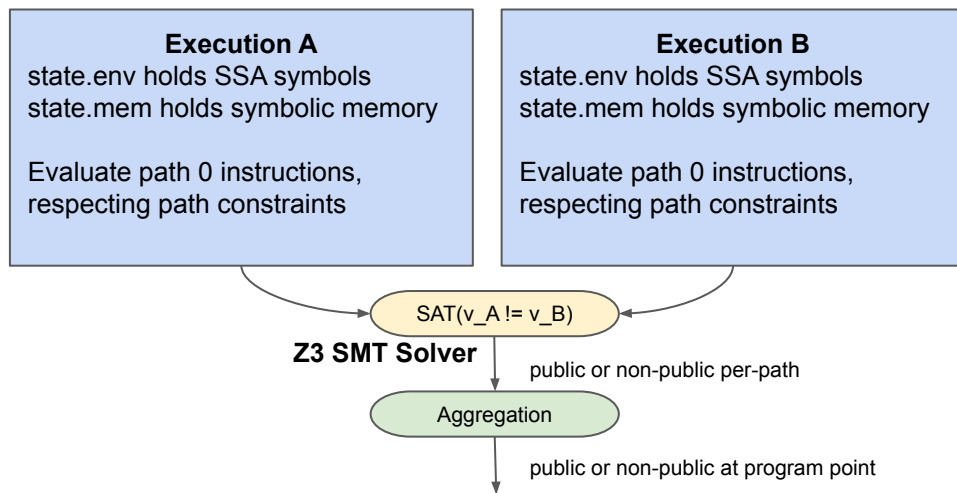
Transmitter Equality:

$\text{load.addr}(v_7)_A == \text{load.addr}(v_7)_B$

$\text{br.cond}(v_{11})_A == \text{br.cond}(v_{11})_B$

$\text{store.addr}(v_0)_A == \text{store.addr}(v_0)_B$

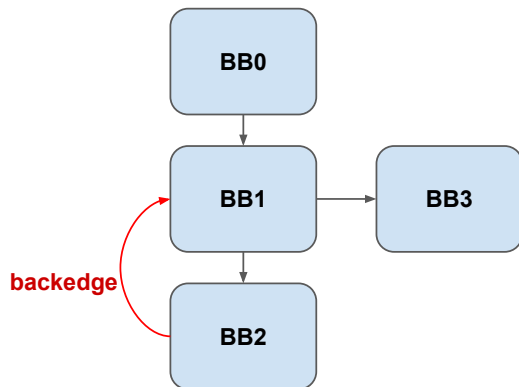
$\text{store.addr}(v_{16})_A == \text{store.addr}(v_{16})_B$



Implementation: Loop Handling

Bounded Paths

- `MAX_LOOP_ITERS = n` enforces bounding, cutting off DFS after `n` revisits
 - `n = 0` → acyclic (stop before revisiting any BB)
 - `n = 1` → allow one revisit
 - `n = 2` → allow two revisits
- Bounded repeated paths reuse the *same* static SSA IDs
 - To prevent contradictions, analyze first iteration to determine publicness for following iterations through loop
 - Heuristic-based rather than fully inductive



Example:

`MAX_LOOP_ITERS = 1`

Bounded Repeated Path 0:

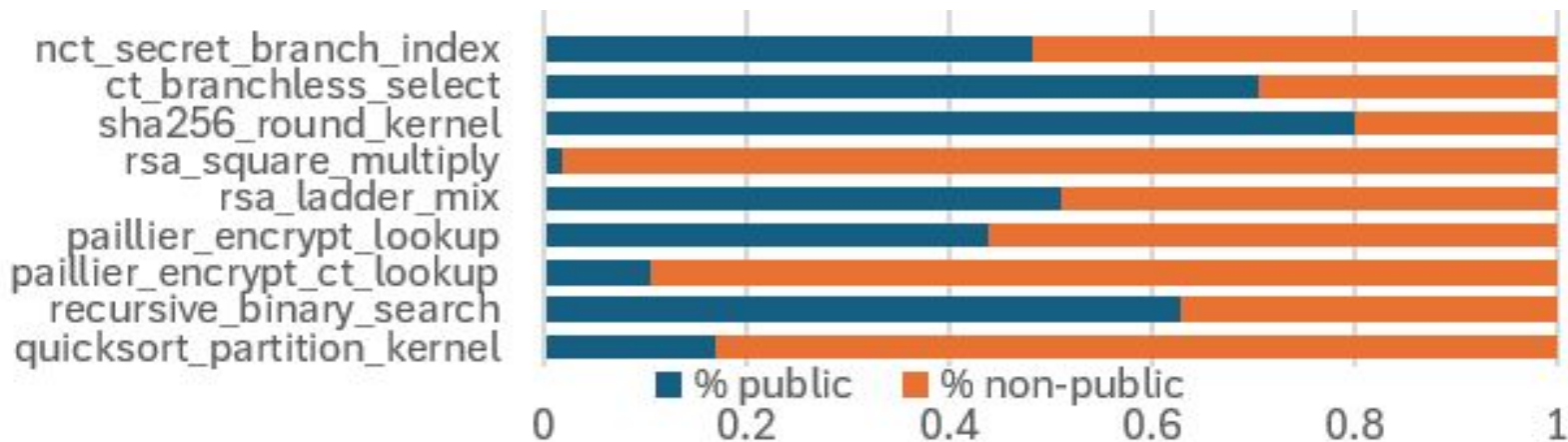
`BB0 → BB1 → BB2 → BB1 → BB3`

Evaluation Methodology

- Evaluate reduced overprotection relative to naive baseline (100% private)
 - Measure number of explored paths, number of SMT queries, sat count (public), unsatisfiable count (not public), unknown count (solver unknown/timeouts), solve check time for function
 - Validate with manual “public” determination analysis since publicness is known by construction
- Benchmark Evaluation Suite
 - 39 functions across 9 source files
 - Sanity check tests (constant time vs. non-constant time, single path vs. multi-path secret-dependent)
 - Control flow (branching, loops, recursion)
 - Classic benchmarks (quicksort partition, travelling salesman)
 - Cryptographic functions (Paillier lookup, square-and-multiply, compression-round kernel)

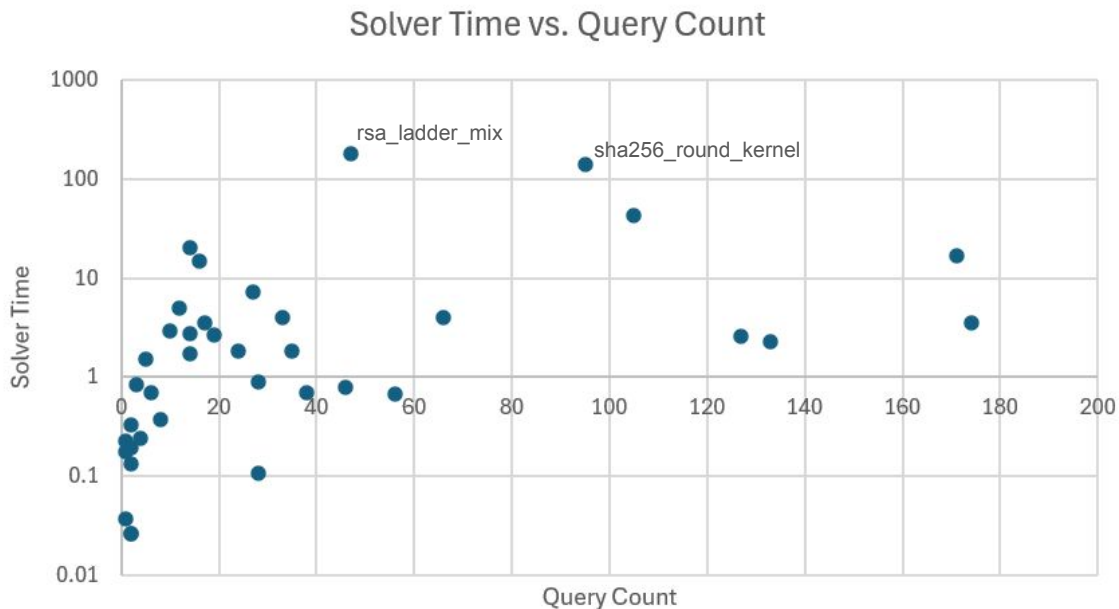
Results

- 39 functions tested with 63 paths enumerated, 609 transmitters identified, and 1388 SMT queries issued to the solver
- Blue indicates potential for reduced overprotection



Results

- Analysis cost dependent on control-flow *and* semantic complexity of constraints
- One-path functions are still computationally expensive



Future Work

Challenges & Limitations

- Loop handling not fully inductive
- Limited evaluation suite

Future Work

- Fully inductive loop handling
- Extend testbench with comprehensive set of cryptographic and security-critical programs
- Integrate project with complementary toolchains
 - Make various parts of program constant-time
 - Automate protections based on LLVM IR publicness analysis

Thank You!

Questions?

Works Cited

- [1] Xiaoxuan Lou, Tianwei Zhang, Jun Jiang, and Yinqian Zhang. 2021. A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptography. *ACM Computing Surveys (CSUR)* 54, 6(2021), 1–37.
- [2] Zhiyuan Zhang, Gilles Barthe, Chitchanok Chuengsatiansup, Peter Schwabe, and Yuval Yarom. 2023. Ultimate {SLH}: Taking speculative load hardening to the next level. In *32nd USENIX Security Symposium (USENIX Security 23)*. 7125–7142.
- [3] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, et al . 2020. Spectre attacks: Exploiting speculative execution. *Commun. ACM* 63, 7 (2020), 93–101.